
NiCE Text Grid Administration Guide Version 20.2.0.2

- [Copyright](#)
- [Deployment Model](#)
 - [Gateway Service](#)
 - [Search Service](#)
 - [Service Startup Order](#)
 - [Automatic Index Data Migration](#)
 - [High Availability](#)
 - [HA Gateway](#)
 - [HA Search](#)
- [Installation and Configuration](#)
 - [Network Ports](#)
 - [Memory Configuration](#)
 - [Watermark Settings](#)
 - [Prohibiting Certain Searches](#)
 - [Deleting Old Indexes \[\[closedIndexExpiryDays\]](#)
 - [Changing the Shard and Replica Counts on Search Nodes](#)
 - [Changing Shard or Replica Count at Initial Configuration](#)
 - [Changing Shard or Replica Count on New Indexes for an Existing Search Node](#)
 - [Modifying Shard Count for Existing Indexes using shrink, reindex, or split.](#)
 - [Modifying Replica Count for Existing Indexes Using SetReplicaCount.ps1](#)
 - [Checking Cluster Status after Adjusting Replica Count](#)
 - [Getting Highlights From Very Large Documents](#)
 - [Validating an Install](#)
 - [Post Validation Cleanup](#)
- [Scaling Out](#)
 - [Gateway Nodes](#)
 - [Search Nodes](#)
 - [Initial Deployment](#)
 - [Tiebreaker Nodes](#)
 - [Adding a Search Node](#)
 - [Removing a Search Node](#)

- [Administration](#)
 - [Changing Servers for an Existing Installation](#)
 - [Restarting a Cluster](#)
- [Secure Communication](#)
 - [Server Certificates](#)
 - [Configuring Secure Communication](#)
 - [Custom Secure Communication](#)
 - [Enabling SSL/TLS for Kibana](#)
 - [Secure Communications Credentials](#)
- [Credentials](#)
 - [Gateway REST Credentials](#)
 - [Configuration settings \(<dataDir>\gateway\etc\application.yml\)](#)
 - [Initial users credentials:](#)
 - [Roles and LDAP groups](#)
 - [Delegating User Authentication to an LDAP Server](#)
 - [nxGridLdap.ldif:](#)
 - [Gateway Windows Integrated Authentication](#)
 - [<dataDir>\gateway\etc\application.yml:](#)
 - [Search Node\(s\) REST Credentials](#)
 - [Security Mode Prerequisite: Installing a Commercial ElasticSearch License](#)
 - [LDAP Based Authentication:](#)
 - [AD Based Authentication:](#)
- [Common Exception Protocol \(CEP\) Server Configuration](#)
 - [A Little About Certificates](#)
 - [A Little About Keystores](#)
 - [Using The Default CEP Client Certificate](#)
 - [Using Custom Certificates](#)
 - [Importing Custom Trusted Root Certification Authority Certificates](#)
 - [Importing a Dual-Use Certificate](#)
 - [Importing Encryption Certificates](#)
 - [Setting CEP Properties In Text Grid](#)
 - [Mandatory Property To Enable CEP](#)
 - [Optional CEP Properties](#)
- [Tools](#)
 - [tg](#)
 - [help](#)
 - [status](#)
 - [remove-searchnode](#)

- [esindices](#)
- [remove-migrated-indices](#)
- [gateway-registry](#)
- [tg2-info.ps1](#)
- [treeconfigurer](#)
 - [Special Considerations for Upgrades](#)
 - [Sensitivity to Format Change](#)
 - [Command Line Mode](#)
 - [Graphical User Interface Mode](#)
 - [Validate Config](#)
 - [Write Config](#)
 - [Install Services](#)
 - [Install Nodes as Windows Services](#)
- [moveTextGrid2DataDir.ps1](#)
- [ValidateInstall.ps1](#)
- [PrepareTG2Upgrade.ps1](#)
- [migrate](#)
 - [Usage](#)
 - [Configuration File](#)
 - [Before installation of Text Grid 20.x](#)
 - [Running the migrate app](#)
 - [Migrate as a Service](#)
 - [Using the idFile property](#)
 - [Optimization](#)
 - [Post Migration Validation](#)
 - [Optional Post migration cleanup](#)
- [mergeNewIdsFile](#)
- [Kibana](#)
 - [Configuration](#)
 - [Starting and running Kibana](#)
 - [Executing a cURL command using Kibana](#)
 - [Implementing Disaster Recovery using Cross Cluster Replication](#)

Copyright

PROPRIETARY AND CONFIDENTIAL INFORMATION

Information herein is proprietary information and trade secrets of NICE Ltd. and/or its affiliated companies (Affiliates). This document and the information herein is the exclusive property of NICE and its Affiliates and shall not be disclosed, in whole or in part, to any third party or utilized for any purpose other than the express purpose for which it has been provided.

IMPORTANT NOTICE

Subject always to any existing terms and conditions agreed between you and NICE or any Affiliate with respect to the products which are the subject matter of this document, neither NICE nor any of its Affiliates shall bear any responsibility or liability to a client or to any person or entity with respect to liability, loss or damage caused or alleged to be caused directly or indirectly by any product supplied or any reliance placed on the content of this document. This includes, but is not limited to, any interruption of service, loss of business or anticipatory profits or consequential damage resulting from the use or operation of any products supplied or the content of this document. Information in this document is subject to change without notice and does not represent a commitment on the part of NICE or any Affiliate.

All information included in this document, such as text, graphics, photos, logos and images, is the exclusive property of NICE or in Affiliate and is protected by United States and international copyright laws. Permission is granted to use, view and photocopy (or print) materials from this document only in connection with the products to which this document relates and subject to the terms of license applicable to such products. Any other use, copying, distribution, retransmission or modification of the information in this document without the express prior written permission of NICE or an Affiliate is strictly prohibited. In the event of any permitted copying, redistribution or publication of copyrighted material, no changes in, or deletion of, author attribution, trademark legend or copyright notice shall be made.

Products supplied may be protected by one or more of the US patents listed at www.nice.com/Patents. For the full list of trademarks of NICE and its Affiliates, visit www.nice.com/Nice-Trademarks. All other marks used are the property of their respective proprietors.

All contents of this document are: Copyright © 2025 NICE Ltd. All rights reserved

For assistance, contact your local supplier or nearest NICE Customer Service Center:

**EMEA Region (Europe,
Middle East, Africa)**

Tel: +972-9-775-3800
Fax: +972-9-775-3000
email: support@nice.com

**APAC Region (Asia/
Pacific)**

Tel: +852-8338-9818
Fax: +852-2802-1800
email: support.apac@nice.com

North America

Tel: 1-800-663-5601
Fax: +201-356-2197
email: na_sales@nice.com

International

Headquarters-Israel
Tel: +972-9-775-3100
Fax: +972-9-775-3070
email: info@nice.com

**The Americas Region
(North, Central, South
America)**

Tel: 1-800-6423-611

Fax: +720-264-4012

email:

support.americas@nice.com

Israel

Tel: 09-775-3333

Fax: 09-775-3000

email: support@nice.com

France

Tel: +33-(0)1-41-38-5000

Fax: +33-(0)1-41-38-5001

Hong-Kong

Tel: +852-2598-3838

Fax: +852-2802-1800

United Kingdom

Tel: +44-8707-22-4000

Fax: +44-8707-22-4500

Germany

Tel: +49-(0)-69-97177-0

Fax:

+49-(0)-69-97177-200

NICE invites you to join the NICE User Group (NUG).

Visit the NUG Website at www.niceusergroup.org; and follow the instructions.

All queries, comments, and suggestions are welcome! Please email: nicebooks@nice.com.

For more information about NICE, visit www.nice.com

Deployment Model

This document describes Text Grid systems in terms of services and nodes. In Text Grid terminology, a *service* is a software component that can be started and stopped, e.g. via the Windows Service Control Panel. A *node* is a role that a server fulfills in a system. For example, a Text Grid Search Node is a server hosting a Text Grid Search Service.

The Search service wraps open source software (OSS) components. Such components are identified in this document when the components' presence has implications for administering a Text Grid system. Please note that supported configurations of such systems are limited to those explicitly described in this document.

Gateway Service

The Gateway Service acts as an API gateway, receiving HTTP requests from client applications and routing them to Search nodes.

A normal Text Grid system requires exactly one Gateway Node.

Search Service

The Search Service supports full-text search by hosting Elasticsearch, an open source distributed search framework. (As a result, some configuration files and log files identify the service as "Elasticsearch" rather than "Search Service".)

Technically, Text Grid does not contain a system of record. If the indexes need to be rebuilt due to a major upgrade, this must be done by re-indexing the documents from source. Thus, any application using Text Grid needs to maintain a system of record of the original documents.

Search Services communicate with each other as well as with the Gateway Services.

Text Grid requires at least one Search Node. Additional Search Nodes may be added according to the procedures in the [Scaling Out](#) section. Adding Search Nodes increases index and search performance, and also spreads index storage across multiple servers.



While Elasticsearch is installed as part of the TextGrid 20.x install, in order to use some features, including AD Authentication, Snapshots, and Cross Cluster Replication, you must install an Elasticsearch License purchased either from NICE or directly from Elasticsearch. For instructions on how to install the license, consult [Security Mode Prerequisite: Installing a Commercial ElasticSearch License](#). Also note that once the license is installed, there are some additional requirements. For instance, either SSL must be enabled on the Search Cluster, or all xpack features must be disabled.

Service Startup Order

Text Grid services on a single server do not need to be started in any particular order. However, to minimize startup time

and unnecessary log messages, the following order is recommended:

1. Search Service(s)
2. Gateway Service(s)

Automatic Index Data Migration

Whenever document mapping or analysis configuration changes are made, some or all existing document indices in an existing deployment site may need be migrated to the new version. The underlying re-indexing operation is kicked off automatically the first time Gateway service is started after the upgrade. Reindexing old documents may take hours if the data set is large, but usually result in more efficient data storage and search performance.

It is important to do a proper backup of the data directory before embarking on the upgrade and allocate enough free disk space (at least 50%), and ensure a long enough maintenance window for better chance of success. Reindexing performance may be affected by a number of factors, of which the most important is the degree of parallel processing, controlled by a special configuration "elasticsearch.settings.reindex.slices" in application.yml. The default value of 12 should be adequate for most cases, but depending on hardware capability, it can be increased or decreased on special occasions. Other parameters related to reindexing include scroll batch size and shard response timeout value, corresponding to gateway configurations "elasticsearch.settings.reindex.batchSize" and "elasticsearch.settings.reindex.timeoutMinutes" respectively. The default values of 1000 and 5 minutes should only be changed in case of a failure as a last resort.

Because the migration reindexing operation is a part of the Gateway service initialization sequence, the service is unreachable by the tg commands while migration is in progress. To check on the work status, closely monitor the Gateway service log, where the percentage completion is indicated, as in the following example:

```
| <timestamp> INFO MIGRATIONPROCESSED: 99.97%
```

Migration completion is indicated by the following line in the Gateway service log:

```
| <timestamp> INFO Migration completed.
```

The log also reports the total documents created and other statistics.

Once the entire TG upgrade has been successfully completed, the old index may be removed to free up disk space with the [remove-migrated-indices](#) command.



When using high availability mode, the migration must be completed on one gateway service before the other gateway services are started. The migration process is not designed to be run in parallel on multiple gateway services.

High Availability

It is possible to deploy Text Grid in a High Availability (HA) configuration by utilizing multiple Gateway Services behind a load balancer, along with multiple Search Services.

HA Gateway

Multiple Gateway Services may be deployed behind a load balancer without requiring any additional services or configuration changes. Each Gateway instance runs independently, requiring no communication between instances.

HA Search

The Search Service may be deployed in a cluster

Installation and Configuration

Before installing, ensure that firewall rules and network configuration allows communication on ports as described in the [Network Ports](#) section.

The installation and configuration process involves up to three steps:

1. If your organization prohibits the use of the bundled JRE, install a Java Server Runtime Environment (JRE).
 - This option is intended only for compliance with policies at specific deployment sites. In general, it is discouraged because the resulting configuration will not have undergone testing.
 - Only Server JRE version 17 with JavaFX bundle is supported. The non-server distribution with browser integration and automatic updates is not supported.
 - Using the Control Panel, create a system environment variable NXTGRID_JAVA_HOME and set its value to the location of the JRE. The JRE location should have a bin/server directory with a jvm.dll in it.
2. Install the system via the Setup-TG.ps1 and associated zip file.
 - Ensure both files are in the same directory
 - As Administrator run `Setup-TG.ps1 --installdir <installationDirectory> --datadrive <datadrive> [ --nosearch ] [ --nogateway ] [ --esksowneroverride ]`
 - The command line parameters in the Setup-TG.ps1 are optional.
 - If no installationDirectory is supplied, the user will be prompted for the install dir.
 - If TextGrid has already been installed and no datadrive is supplied, the previous datadrive will be used.

- As of 12.4.2, the datadir is ALWAYS <dataDrive>\ProgramData\Nexidia\TextGrid2. If the previous datadir does not match this format, you must first run the [moveTextGrid2DataDir.ps1](#) script to move the datadir to a location that matches this pattern.
- If TextGrid has not already been installed and no datadrive is supplied, the user should enter the datadrive on the Directories Node of the TreeConfigurer.
- If this node is only going to run Gateway and not Search, or Search and not Gateway then the --nosearch or --nogateway parameters can be used, or the service can be disabled within the TreeConfigurer. The --nosearch, --nogateway, and --nokibana parameters are only effective on initial install, and are ignored on all upgrades and subsequent configuration changes, so that a user doesn't have to remember to pass them each time treeconfigurer is used to update another parameter.
- Elasticsearch added elasticsearch.keystore file owner validation in recent release, this validation could cause search service start up failed if the service account being used to run search service is also an administrator of search server. --esksowneroverride flag could be used to resolve this problem.
- If the password associated with the service account running search service contains special characters that the install cannot handle, then when you try to start the service, it will not start and a popup dialog shows Error 1069: The service did not start due to a logon failure. To fix, re-enter the password in the Windows services popup "Log on" tab before starting the search service. Once corrected, this does not need to be repeated the next time the service is started.
- Any other command line options will be passed through to the treeconfigurer (e.g. --nogui)
 - After installation, the treeconfigurer app will automatically run
 - Within the treeconfigurer application, make any configuration changes, save them, and install the Windows services
 - Note that installation is not complete unless the treeconfigurer application completes successfully.

3. Start the TextGrid 2.0 Search and TextGrid 2.0 Gateway Windows Services.

Network Ports

All ports use TCP traffic unless otherwise noted.

Gateway Node

- 28002: HTTP/HTTPS

Search Node

- 9200: HTTP

- 9300: Transport

Memory Configuration

The default memory for Search Nodes is 30G. This is a realistic value for most deployments. However, in some scenarios such as a test server that is being used for Text Grid and other services, you may want to reduce this memory footprint. This is done in the treeconfigurer on the Search Heap Size page or by passing a `--heapsize` parameter to the treeconfigurer. In all cases, the value is in Gig (so 30 == 30 Gig).

Watermark Settings

Elasticsearch (which is used by the Text Grid Search Node) protects and warns about low disk space situations based on the following defaults:

Elasticsearch Cluster Setting	Default Value	Log Level
cluster.routing.allocation.disk.watermark.low	85%	INFO
cluster.routing.allocation.disk.watermark.high	90%	WARN
cluster.routing.allocation.disk.watermark.flood_stage	95%	ERROR

The low watermark indicates that Elasticsearch will not allocate shards to nodes that have more than (in this case) 85% disk used. It can also be set to an absolute byte value (like 500mb) to prevent Elasticsearch from allocating shards to a node where less than 500mb of disk space is available. The high watermark is used to tell Elasticsearch to relocate shards away from a node whose disk usage exceeds the given percentile (or has less than the provided byte value, similar to the low watermark setting). The flood stage setting is used by Elasticsearch as a defense mechanism to prevent nodes from running out of space and is used to enforce a read-only index block on all indices that have one or more shards allocated on that node where the disk space has exceeded the given percentile (or has less than the provided byte value, similar to the low and high watermark settings). If the flood stage has been activated, and the indexes have a block, the block will eventually be automatically released once space is made available, but if you want to release it immediately, instructions are at <https://www.elastic.co/guide/en/elasticsearch/reference/current/fix-watermark-errors.html>. KB0076993 provides some additional information on this topic.

It is discouraged as a bad practice to install Text Grid onto a shared hard drive where it would only occupy a small portion of a much larger disk. However, if that is a requirement or constraint for a particular installation then these defaults may need adjusting. For example, Text Grid could be deployed onto a 14TB drive, so if 95% of 14TB is occupied on disk, that still leaves 700GB worth of disk space. For these cases, tweak those cluster settings in order to provide more reasonable thresholds.^[1] As for values, we can either provide a percent value as shown in the table, or an absolute byte value (like 500mb or 20GB), but the same unit of measurement should be used for all values. You cannot

mix and match percentiles with byte values amongst the different settings.

It is important to note here that Elasticsearch has two different mechanisms for updating its settings, via the `elasticsearch.yml` file or by using the cluster settings update api.^[2] In the case where a setting is set in both locations, Elasticsearch will use the value stored in its cluster settings over the value in the local `elasticsearch.yml` file.^[3] The rationale here being that the `elasticsearch.yml` file should be used for local settings, either pertaining to the individual node itself or aid in its ability to join the cluster, whereas the cluster settings apply to the entire cluster of nodes. Note that when using the `elasticsearch.yml` to assign Elasticsearch settings, the Text Grid search service (Elasticsearch) must be restarted for the settings to take effect. However, when using the cluster settings update api, the changes take place immediately without having to restart the nodes in the cluster.

Instructions for changing these settings are at <https://www.elastic.co/guide/en/elasticsearch/reference/current/fix-watermark-errors.html>.

Prohibiting Certain Searches

There are certain searches that may be OK to use on small datasets, but can be extremely time intensive on larger ones. These include wildcard searches, particularly ones with either no or a short prefix (e.g. "little * hen" is much slower than "little r* hen") and fuzzy searches, where the prefix length is 0 or short. The default install prohibits searches without at least 2 characters of prefix and fuzziness. These restrictions can be tightened or loosened by setting the "nexidia.tg.query.minFuzzinessPrefix" and "nexidia.tg.query.minWildcardPrefix" to other values. These attributes are set in both the `elasticsearch.yml` file (which controls actual searching) and the `gateway application.yml` file (which controls the results from the `structuredqueryvalidation` endpoint).

Deleting Old Indexes `[[closedIndexExpiryDays]`

When an index is no longer needed due to migration, it can be deleted using the [remove-migrated-indices](#) command. This command will remove all indexes that have been migrated to the new version of Text Grid. This command is only available on the Gateway Node.

By default, when the Gateway service starts up, it checks, and any closed index that is older than 1 month will be deleted. In the `application.yml` file, the `nexidia.tg.index.closedIndexExpiryDays` property can be set to change this one month period. If the value is negative, closed indices will not be deleted. If the value is positive, then it is the number of days that a closed index must be older than to be deleted. Zero would mean to delete without checking the age.



This setting can **NOT** be set in `treeconfigurer`, as this value does not normally need to be modified.

Changing the Shard and Replica Counts on Search Nodes

Elasticsearch breaks the indexes into elements known as shards that are distributed across multiple nodes. For initial

deployment, the shard count is controlled by the `elasticsearch.settings.shards` in the gateway `application.yml` file.

Elasticsearch will make copies of the data in multiple places so that if one node is busy, the same data can be found in another one. This is called the ReplicaCount. A ReplicaCount of 1 would mean that the data will be in the cluster twice, once as an original, and once as a replica. For initial deployment, the replica count is controlled by the `elasticsearch.settings.replicas` key in the gateway `application.yml` file and is 0. This value can be modified using the `--replicas` parameter to `treeconfigurer` or using the `treeconfigurer` application.

Changing Shard or Replica Count at Initial Configuration

After install but before starting the services, to change the number of shards to 16, and the number of replicas to 1, edit `<data-dir>\gateway\etc\application.yml` to replace the values in the follow lines:

```
elasticsearch:
  settings:
    shards: '16'
    replicas: '1'
```



This has no effect on any indexes that already exist.



16 and 1 are used strictly for illustration and are not necessarily the recommendation for a particular install.

Changing Shard or Replica Count on New Indexes for an Existing Search Node

To modify these settings for indexes that do not yet exist, when documents do already exist (e.g. you already have general documents, but you want to change the shard count for chat documents, none of which have yet been loaded):



You can determine which indexes already exist using the `esindices` command.

1. Stop the `NICE Text Grid 2 Gateway` service.
2. Stop the `NICE Text Grid 2 Search` service on all Search Nodes.
3. Modify the `application.yml` document as described [above](#).
4. Restart the two services.

Any indexes (for new languages or types) that are created will have the new shard and replica count. Any existing indexes will have the old shard and replica count.

Modifying Shard Count for Existing Indexes using shrink, reindex, or split.



This is an advanced operation, requiring reading and understand the referred links below.

To modify the shard count on all existing (and future) indexes:



If you wish to change the replicaCount as well as the shardCount, that can be done as part of this process.

1. Stop access to Text Grid.
2. Stop the `NICE Text Grid 2 Gateway` service.
3. Stop the `NICE Text Grid 2 Search` service on all Search Nodes.
4. Modify the `application.yml` document as described [above](#).
5. Restart the two services.
6. For each index:
 - a. use one of the following Elasticsearch commands to re-shard the index into a new temporary index:
 - i. If the destination number of shards is smaller and equally divisible (e.g. starting with 12, you are going to 2,3,4 or 6 shards) use the [Shrink API](#).
 - ii. If the destination number is larger, and a multiple of the original number of shards (e.g. going from 2 to 6 shards), then use the [Split API](#).
 - iii. For any other re-sharding, use the [Reindex API](#).
 - b. Ensure that the re-sharding is complete by waiting for the index to go green.
 - c. Use the [Delete API](#) to delete the original index.
 - d. Use the [Clone Api](#) to rename the temporary index to the original index name.
 - e. Wait for the original index to go green
 - f. Use the [Delete API](#) to delete the temporary index.
 - g. set the alias back to this index (e.g. if `nxtg_general_en_4_3` has been re-sharded, then set an alias `nxtg_general_en` to `nxtg_general_en_4_3`) using the [Alias API](#)
7. Restart access to Text Grid.

Modifying Replica Count for Existing Indexes Using SetReplicaCount.ps1

Note that it is not possible to modify shardCount on existing indices without copying to another index as described above. However, if replicaCount is the only value that needs modification, this method will not take the system down for as long as the reindex method for Search Nodes clusters with a large number of documents that are already indexed.

To modify the replica count on all existing (and future indexes):

1. Stop the `NICE Text Grid 2 Gateway` service.
2. Stop the `NICE Text Grid 2 Search` service on all Search Nodes.
3. Modify the `application.yml` document as described [above](#).
4. Restart the two services.
5. Run `SetReplicaCount.ps1` (passing the desired ReplicaCount).

Checking Cluster Status after Adjusting Replica Count

When the number of replicas in the cluster is not up to the required replicaCount, the status of the indices in the cluster will be yellow. Yellow means that searches will still work, but the Search cluster is still adjusting to meet the replica requirement. Once the cluster has completed the adjustments, the status will go green.

Use the [esindices](#) command to get an instantaneous view of the status of the indices.

For the very specific case of taking a single Search Node system and adding a second Search Node as a replica (e.g. `replicaCount == 1`), the `CheckReplicaStatus.ps1` script will provide a progress bar on the transition from a single node to dual node system.



In order to ensure that replicas have been correctly set up for each index, run the [tg esindices](#) command and check the replica count column for the indices.

Getting Highlights From Very Large Documents

When a document is very large, searches that should highlight that document will return a 413 (PAYLOAD_TOO_LARGE) error.

```
{
  "timestamp": "2024-05-19 02:26:17.024",
  "code": "payload_too_large",
  "resource_uri": "http://10.1.47.139:28002/search",
  "exception": "TgTooLargeException",
  "message": "The field you are trying to highlight is too large to be analyzed. Please check ntextgrid.log and set max_analyzed_offset",
  "errors": {
    "details": "The field you are trying to highlight is too large to be analyzed. Please check ntextgrid.log and set max_analyzed_offset",
    "cause": null,
    "stackPreview": [
      {
        "classLoaderName": "app",
        "moduleName": null,
        "moduleVersion": null,
        "methodName": "buildDocumentSearchImpl",
        "lineNumber": 153,
        "nativeMethod": false,
        "className": "com.nexidia.grid.text.service.impl.document.DocumentSearchImpl"
      },
      {
        "classLoaderName": "app",
        "moduleName": null,
        "moduleVersion": null,
        "methodName": "search",
        "fileName": "..."
      }
    ]
  }
}
```

```
java", "lineNumber": 71, "nativeMethod": false, "className": "com.nexidia.grid.text.service.impl.document.DocumentSearchImpl"}]]}]}
```

Check the `nxtxtgrid.log` for a log such as

```
[2024-05-19T02:26:16,859][ERROR][c.n.t.e.h.NxTgHighlighter] [null] Error highlighting field The length [1257291] of field [content] in doc[0]/index[nxtg_general_en_4_3] exceeds the [index.highlight.max_analyzed_offset] limit [1000000]. To avoid this error, set the query parameter [max_analyzed_offset] to a value less than index setting [1000000] and this will tolerate long field values by truncating them.
```

In order to fix this error, run `SetMaxAnalyzedOffset.ps1`. Specify the correct `docType` and language, and pass an offset length greater than the length of your large document. If no offset length is set, the maximum length is 1000000.

The number you want to choose as the new `max_analyzed_offset` is a number slightly larger than the number (1257291 in the above example) in the `nxtxtgrid.log`.



The error log message is somewhat confusing, because it says to "set the query parameter `[max_analyzed_offset]` to a value **less** than index setting `[1000000]` and this will tolerate long field values by truncating them".

However, the `setMaxAnalyzedOffset.ps1` script is NOT setting this parameter on a single query, as the log recommends, but is setting it for the entire index. This index based setting must be larger as it does not do the truncation.

Validating an Install

After installing and starting the services, run the [ValidateInstall.ps1](#) script to do a quick sanity check to confirm the install works. If the credentials for the elasticsearch service user are stored in the Windows Credential Manager, then use the `-esauth` parameter to pass this info (e.g. `ValidateInstall.ps1 -esauth "gateway:gateway"`)



When restarting a cluster, it is important that the master node be up and running before other nodes are booted up. Therefore, please start, or restart, master before all others.

Post Validation Cleanup

Once the `ValidateInstall` script passes, run the `PostValidationCleanup.ps1` script on all the search nodes. This script eliminates the `cluster.initial_master_nodes` section from the `elasticsearch.yaml` file. This script is not necessary for a single-node system, since the `cluster.initial_master_nodes` section is not used in that case.

Scaling Out

The various Text Grid node types have different capacities, motivations, and mechanisms for deploying more than one node of the same type.

Gateway Nodes

Multiple Gateway Nodes may be deployed for High Availability (HA). Because Gateway doesn't do any real processing itself, additional Gateway Nodes will not appreciably increase performance.

Multiple Gateway Nodes may be deployed behind a load balancer without requiring any additional services or configuration changes. Each Gateway Node runs independently, requiring no communication between nodes. Gateway Nodes may be added or removed at any time, without the need for configuration changes.

Search Nodes

Multiple Search Nodes may be deployed for both High Availability (HA) and performance. Search Nodes coordinate communication between themselves when properly configured. No load balancer is required when multiple Search Nodes are deployed.

Configuring a Search Node cluster involves choosing which node will act as a *seed*. The seed node helps Search Services discover one another. When an Elasticsearch service starts, it contacts a seed to advertise information about itself and to find out what other nodes are in the cluster. Please only use a single seed node.

Initial Deployment

When deploying a new Text Grid system with multiple Search Nodes, the Seed Node and Master Nodes must be set in the Search Cluster in the Treeconfigurer.

Set the seed node to the network bind addresses. Also, set the Master Nodes to the set of node names (default to host names) that are master eligible.

The resulting elasticsearch.yml file will have entries similar to:

```
discovery.seed_hosts:
- 10.0.2.3
cluster.initial_master_nodes:
- es-node1
- es-node2
```

In any Search Node system that does not include the Gateway Node on the same system, set the Gateway Server attributes for the TG Admin in the TreeConfigurer to point to the Gateway Node.

Ideally, *all* Search Nodes in the system should be configured to use the *same* value for the seed node.

Tiebreaker Nodes

Elasticsearch recommends a setup of two nodes that process data and one dedicated node for master elections as the smallest possible cluster size for production setups. This allows for data processing to continue even if one of the nodes goes down. In order to set up a voting only node, click the check box on the Search Cluster menu in Tree Configurer.

This will make the search node a dedicated voting only node. It will not process any data, so fewer resources can be allocated to the server, and will focus on cluster stability.

Adding a Search Node

1. Install Text Grid on the new nodes, but do not start the services yet. If one or more of the new nodes are intended to serve as a seed or initial master mode, update the `elasticsearch.yml` file appropriately at each Search Node before restarting the Search Service. Configure the new nodes to use the same settings as others.
 2. Start all previously installed Search Services.
 3. Start each of the new Search Services. As they join the cluster, the cluster will automatically begin relocating data to the new nodes.
 4. Update the Gateway Service configuration `elasticsearch.cluster-nodes.host` in `application.yml` as appropriate before re-starting the service.
 5. Update the `tgadmin.yml` file to set the `gateway.address` value to point to the appropriate Gateway Node.
-

Removing a Search Node

1. From any Search Node, use the `<install location>\apps\bin\tg` application to run the `remove-searchnode` command, passing the IP addresses of each node to be removed. Data will immediately begin relocating to other nodes. For example:

```
tg remove-searchnode 10.0.2.3 10.0.2.4
```

Be aware that issuing the `remove-searchnode` command a second time will negate the previous command. If a different set of IP addresses is given, data will immediately begin relocating away from the new set of nodes, and the previous set of nodes will begin receiving data.

2. Use the `--status` argument to the `remove-searchnode` command to monitor the progress of data relocation. Provide as arguments the IP addresses of the nodes being removed. For example:

```
tg remove-searchnode --status 10.0.2.3 10.0.2.4
```

3. Monitor progress by repeating step 2 until data has been relocated from *all* Search Nodes to be removed.
 4. If any of the removed nodes were seed node, remove it from the `discovery.seed_hosts` in the `elasticsearch.yml` file on every Search Node that is being retained. Make sure to add a new seed node in the 'elasticsearch.yml'.
 5. If necessary, update the `cluster.initial_master_nodes` setting in the `elasticsearch.yml` file on every Search Node that is being retained.
-

6. If any changes were made for steps 4 or 5, restart the Search Service on every Search Node that is being retained.
7. Stop the Search Service on the nodes being removed.
8. Update the Gateway Service configuration `elasticsearch.cluster-nodes.host` in `application.yml` as appropriate before re-starting the service.

Administration

Changing Servers for an Existing Installation

If it becomes necessary to move the installation of Text Grid to a new server, follow the following steps:

- Completely shut down the text grid services in all nodes.
- Install the exact same version on the new server with exact services selected.
- If the new server has a new IP, set that IP correctly on all the other servers that reference it.
- Delete contents of the data directory [something like `G:\ProgramData\Nexidia\TextGrid2`] .
- Copy contents of the data directory from the original grid that is being migrated (may use robocopy or other utility).
[Verify that service user has read\write permission in the new location]
- Ensure file count and sizes match.
- Repeat the steps for other servers (if relocating multiple servers).
- Run treeconfigurer to set any changed IPs and validate the properties are present and correct.
- Start services on the new grid server, and on the other nodes (that did not move). Follow the service starting guideline from [Service Startup Order](#startuporder).
- Open tg-admin-gui and check status. All nodes should be there. Check [esindices](#).
- For large data sets, keep an eye on the logs and give some time. Cluster health may start [RED] and then move to [GREEN].

Restarting a Cluster

Whenever changes are made and a cluster is restarted, the order in which the nodes are stopped and started matters. This is why the default startup for the services is manual.

Perform the following steps on the gateway node of the cluster to make use of scripts included with TG to help facilitate the restart process.

1. Run the `pre_full_custer_restart_tg2.ps1` script.
2. Stop all services
3. Start any search services on the gateway nodes. *Give the service at least a minute to start up before moving onto the next node.*
4. Start any search services on the data nodes. *Give the service at least a minute to start up before moving onto the next node.*
5. Run `post_full_custer_restart_tg2.ps1` script.
6. Start all other Text Grid services on all nodes

Secure Communication

Text Grid can be configured to use secure communication with public-facing services as well as internal communication between services. Security is handled using Secure Sockets Layer (SSL) / Transport Layer Security (TLS).

To utilize secure communications, a server certificate is required. It can be either a certificate signed by a Certificate Authority, or a custom certificate that is self-signed.

Server Certificates

Text Grid uses the standard PKCS#12 (.p12) data format for the key and trust store files configured for the Text Grid search nodes (Elasticsearch cluster) and the Gateway service. A PKCS#12 file is an archive of cryptography objects, mainly X.509 certificates and corresponding private keys. The PKCS#12 file is natively supported by Java and can be managed with the Java keytool utility. Other tools that can be used include the `elasticsearch-certutil` program, OpenSSL, etc. Consult their respective public documentation for details of their usage.

An example key store file (`nxtg.p12`) is included in the Text Grid distribution. It was generated using the ElasticSearch utility and signed by the private Elasticsearch CA. In a production installation this file needs to be replaced by a store file that contains an official certificate or equivalent whereby a chain of trust can be established.

Configuring Secure Communication

For distributed Text Grid deployments, the simplest approach is to create a single file containing a site certificate for all service nodes (search and gateway) and configure it for both the keystore and truststore. This is illustrated with examples configurations referencing the `nxtg.p12` file. By default, SSL/TLS is disabled. To enable it, use the `treeconfigurer` [SSL](#) nodes. These will allow you to set up cert files that will be used for SSL/TLS communications for

both the gateway and search nodes.

The trust store for Text Grid search nodes (Elasticsearch cluster) needs to include CA certificates used to sign certificates being used by Text Grid.

Certificate verification mode (only applicable for secure communication among the elasticsearch nodes) defines how to verify the certificates presented by the elasticsearch nodes. There are three verification modes:

- full - validates on issue date, trusted Certificate Authority (CA), hostname or IP address that matches the names within the certificate
- certificate - the default value, validates on it is signed by a trusted authority (CA), but does not check the certificate hostname
- none - no certificate validation

Custom Secure Communication

If you want to do a more custom security configuration (e.g. SSL/TLS for gateway, but no security for Search), the Treeconfigurer provides appropriate SSL tabs for each service which will need to be set.

The configuration files and cert (p12) files are installed in the `$DATADIR/search/config`, `$DATADIR/gateway/etc`, and `$DATADIR/apps/etc` directories.

The following table shows what will be set in each respective file for each feature:

Table 1. Table Custom SSL Settings

Feature	application.yml	elasticsearch.yml	elasticsearch.keystore	kibana.yml	kibana.keystore
HTTPS Gateway Support	<pre>server: ssl: enabled: true trust-store: nxtg.p12 trust-store-password: key-store: nxtg.p12 key-store-password:</pre>				

Feature	application.yml	elasticsearch.yml	elasticsearch.keystore	kibana.yml	kibana.keystore
HTTPS from Gateway to Search	<pre> elasticsearch: cluster- nodes: - scheme: https </pre>	<pre> xpack. security. enabled: 'true' xpack. security. http.ssl. enabled: 'true' xpack. security. http.ssl. keystore. path: nxtg. p12 xpack. security. http.ssl. truststore. path: nxtg. p12 </pre>	<pre> Unique keys need to be stored in elasticsearch. keystore: xpack. security.http.ssl. keystore. secure_ password xpack. security.http.ssl. truststore. secure_ password </pre>		
HTTPS between Search Nodes		<pre> xpack. security. enabled: 'true' xpack. security. transport. ssl. enabled: 'true' xpack. security. transport. ssl. keystore. path: nxtg. p12 xpack. security. transport. ssl. truststore. path: nxtg. p12 </pre>	<pre> Unique keys need to be stored in elasticsearch. keystore: xpack. security. transport.ssl. keystore. secure_ password xpack. security. transport.ssl. truststore. secure_ password </pre>		

Feature	application.yml	elasticsearch.yml	elasticsearch.keystore	kibana.yml	kibana.keystore
HTTPS between Search Nodes and Kibana		<pre>xpack. security. transport. ssl. verification_ mode: certificate</pre>		<pre>server.ssl.enabled: 'true' server.ssl.keystore. path: nxtg.p12 server.ssl.truststore. path: nxtg.p12 elasticsearch.ssl. keystore.path: nxtg. p12 elasticsearch.ssl. truststore.path: nxtg. p12 elasticsearch.ssl. verificationMode: certificate elasticsearch.hosts: - https://localhost:9200</pre>	<pre>Uniq need store kiban keys se keys pass se trust pass elast ssl.k pass elast ssl.tr pass elast user elast pass</pre>

There are two ways to store passwords for keystore|truststore in elasticsearch.keystore

- Option 1: Use treeconfigurer app.
- Option 2: Use windows command console to run elasticsearch-keystore command to store keystore|truststore password in elasticsearch.keystore file.

```
1. Enable RunAs service account which is running TextGrid Service if the user executes below steps is not the
same user as the service account:
In command line console with administrator right:
runas /user:<domain>\<service account name> cmd
```

2. Set environment variable ES_PATH_CONF in new command line console with runas user
set ES_PATH_CONF=<textgrid data folder>\search\config
3. List what is inside elasticsearch.keystore
<TG2 installation folder>\search\bin\elasticsearch-keystore list
4. Add key/value pair to elasticsearch.keystore with below command, a prompt asks for password.
<TG2 installation folder>\search\bin\elasticsearch-keystore add <unique key>
5. Remove key from elasticsearch.keystore
<TG2 installation folder>\search\bin\elasticsearch-keystore remove <unique key>

Storing secure settings in kibana.keystore is similar to storing them in elasticsearch.keystore.

1. Enable RunAs service account running the TextGrid Service (if the current user is not the same user as the service account):
In command line console with administrator rights:
runas /user:<domain>\<service account name> cmd
2. Set environment variable KBN_PATH_CONF in new command line console with runas user
set KBN_PATH_CONF=<textgrid data folder>\kibana\config
3. List what is inside kibana.keystore
<TG2 installation folder>\kibana\bin\kibana-keystore list
4. Add key/value pair to kibana.keystore with below command, a prompt asks for setting.
<TG2 installation folder>\kibana\bin\kibana-keystore add <unique key>
5. Remove key from kibana.keystore
<TG2 installation folder>\kibana\bin\kibana-keystore remove <unique key>

Enabling SSL/TLS for Kibana

Most of the configuration for SSL/TLS for Kibana can be done using Tree Configurer. However, there are a few settings that need to be set manually in the kibana.keystore file.

With SSL/TLS enabled, Kibana requires a username and to authenticate to Elasticsearch, which are the `elasticsearch.username` and `elasticsearch.password` secure settings. These settings are stored in the kibana.keystore file.

The username is `kibana_system`, one of the built-in users in Elasticsearch. The password has to be set using the following steps:

1. Ensure that security is turned on in Elasticsearch and that the search service is running.
2. Enable RunAs service account which is running TextGrid Service if the user executes below steps is not the same user as the service account:
In command line console with administrator right:
runas /user:<domain>\<service account name> cmd

3. Set environment variable ES_PATH_CONF in new command line console with runas user.
set ES_PATH_CONF=<textgrid data folder>\search\config
4. Reset the password for the built-in user `kibana\_system` in Elasticsearch.
<TG2 installation folder>\search\bin\elasticsearch-reset-password -u kibana_system
5. The utility will ask if it's ok to print the password to console. Type 'y' and press enter.
6. The password will be printed to console. Using the steps above, add it to the kibana.keystore file.

After the `elasticsearch.username` and `elasticsearch.password` are set in the `kibana.keystore` file and Kibana is started and initialized, the user will need a username and password to login.

The username is `elastic`, another one of the built-in users in Elasticsearch. The password is obtained in a similar way to the `kibana_system` password:

1. Ensure that security is turned on in Elasticsearch and that the search service is running.
2. Enable RunAs service account running the TextGrid Service (if the current user is not the same user as the service account):
In command line console with administrator rights:
runas /user:<domain>\<service account name> cmd
3. Set environment variable ES_PATH_CONF in new command line console with runas user.
set ES_PATH_CONF=<textgrid data folder>\search\config
4. Reset the password for the built-in user `kibana\_system` in Elasticsearch.
<TG2 installation folder>\search\bin\elasticsearch-reset-password -u elastic
5. The utility will ask if it's ok to print the password to console. Type 'y' and press enter.
6. The password will be printed to console. Enter this when prompted to login at the Kibana login screen.



The `elasticsearch-reset-password` utility **only** works when security is enabled in Elasticsearch and the Elasticsearch service is running.

Secure Communications Credentials

The password for both the keystore and certificate used for secure communications can be changed in three steps. First, use `keytool` as shown to change the password on a copy of the keystore file itself. Note that Text Grid requires that the password for all certificates in keystore be the same as the password for the keystore itself.

```
keytool -storepasswd -keystore keystore.jceks -storetype JCEKS [-storepass my_old_password -new my_new_password]

keytool -keypasswd -alias my_certificate_alias -keystore keystore.jceks -storetype JCEKS [-storepass my_new_password -keypass my_old_password -new my_new_password]
```

The `-storepass`, `-keypass`, and `-new` options are useful for scripting. If omitted, you will be prompted for the old and new passwords.

Second, the configuration for the various Text Grid services will need to be updated with the new password, and the modified copy of the keystore will need to be installed in each of the services. See [Secure Communication](#) for details on updating these files.

Third, all Text Grid services will need to be restarted for the changes to take effect.

Credentials

Credentials used internally by Text Grid to authenticate access to certain components can be changed as needed.

Gateway REST Credentials

All REST services are authenticated using the HTTP Basic scheme as defined in RFC 2617 (Franks et al. 1999). There are four types of users:

- `admin` - The administrative user has the ability to perform system maintenance activities. There is only one admin user.
- `es_client` - This internal role is reserved for granting search node access from Gateway service.
- `monitoring` - A user can be provided the ability to use the actuator endpoints, but not have access to searching and indexing
- `normal` - An application user that performs typical activities such as indexing and searching. There can be many normal users.

Users are managed using embedded LDAP, external LDAP or AD, the details of which are described below. Note that if neither LDAP nor AD is enabled, or if both are enabled, then the gateway will fail to start with a `TgBadConfigurationException`.

Configuration settings (<dataDir>\gateway\etc\application.yml)

To disable embedded LDAP (for both external LDAP and AD):

```
spring:
# 1) comment out the spring ldap.embedded group.
# ldap:
#   embedded:
#     ldif: classpath:nxGridLdap.ldif
#     base-dn: dc=nexidia,dc=com
#     port: 8389
```

```
# 2) set management.health.ldap.enabled to false
management:
  health:
    ldap:
      enabled: false
```

To change from the embedded LDAP server to an external one:

make sure the external host can be accessed by all gateway nodes.

```
# 1) set security.ldap.url to the appropriate external server
security:
  ldap-authentication:
    enabled: true
  ldap.url : ldap://externalhost:8389/ou=grid,dc=nexidia,dc=com
```

Exactly one of the authentication switches for LDAP vs AD (Windows Authentication) must be true:

```
security:
  ldap-authentication:
    enabled: true
  windows-authentication.enabled: false
```

Initial users credentials:

```
User: admin
Password: admin
Description: Text Grid Administrator
```

```
User: ntxtgrid
Password: ntxtgrid
Description: Text Grid Service (same privileges as a normal user)
```

```
User: defaultuser
Password: defaultuser
Description: Text Grid Client User
```

```
User: prom
Password: prom
Description: Text Grid Monitoring User
```

```
User: gateway
Password: gateway
Description: Text Grid Elasticsearch Client Internal User
```

Roles and LDAP groups

Both Gateway and Search Services have the option of delegating user access control (user authentication and credentials management) to an outside LDAP directory service. However, the rules governing privileges of authenticated

users (authorization) remain within Text Grid. The rules are role-based. The roles are mapped to the following LDAP groups:

Table 2. Table Text Grid User Roles

LDAP Group	Description	Members (built-in users)
cn=USER, ou=grid,dc=nexidia,dc=com	Regular users, including users representing Text Grid client systems	nxttextgrid, defaultuser, admin
cn=ADMIN, ou=grid,dc=nexidia,dc=com	Super users with full access privileges	admin
cn=ACTUATOR, ou=grid,dc=nexidia,dc=com	Monitoring users	prom, admin
cn=ES_CLIENT, ou=grid,dc=nexidia,dc=com	Internal role reserved for granting/obtaining search node access from Gateway service.	gateway

Additional users can be added via standard LDAP data entry management procedure; but the set of groups/roles is fixed, as they constitute part of the Text Grid domain business model.

Delegating User Authentication to an LDAP Server

As initially installed, Gateway access control uses the Spring embedded LDAP server that maintains its user store in an LDIF file (deployed as part of the gateway jar in Text Grid.) In a more security sensitive production scenario, using a standalone LDAP service is recommended. See [Configuration settings](#) for more details.

The `nxGridLdap.ldif` file embedded in the Gateway service is copied below. This can be used as a basic template for the data entries to be added into the external directory service.

Note that if you are going to use an external ldif file, the `ldap.embedded.ldif` field must use a properly formed `file:` uri. For example, if the `<datadir>` is `"C:/ProgramData/Nexidia/TextGrid2"`, then the `application.yml` file would be modified as follows:

```
ldap:
  embedded:
    port: '8389'
    ldif: 'file:///C:/ProgramData/Nexidia/TextGrid2/gateway/etc/nxGridLdap.ldif'
    base-dn: dc=nexidia,dc=com
```

nxGridLdap.ldif:

```
dn: dc=nexidia,dc=com
objectClass: top
objectClass: domain
dc: nexidia

dn: ou=grid,dc=nexidia,dc=com
objectClass: organizationalUnit
ou: grid

#
# Text Grid Roles
#
dn: cn=USER, ou=grid,dc=nexidia,dc=com
objectClass: groupOfUniqueNames
cn: USER
uniqueMember: uid=defaultUser,ou=grid,dc=nexidia,dc=com
uniqueMember: uid=nxttextgrid,ou=grid,dc=nexidia,dc=com
uniqueMember: uid=admin,ou=grid,dc=nexidia,dc=com

dn: cn=ADMIN, ou=grid,dc=nexidia,dc=com
objectClass: groupOfUniqueNames
cn: ADMIN
uniqueMember: uid=admin,ou=grid,dc=nexidia,dc=com

dn: cn=ACTUATOR, ou=grid,dc=nexidia,dc=com
objectClass: groupOfUniqueNames
cn: ACTUATOR
uniqueMember: uid=admin,ou=grid,dc=nexidia,dc=com
uniqueMember: uid=prom,ou=grid,dc=nexidia,dc=com

#
# Dedicated group/role for granting ES access.
#
dn: cn=ES_CLIENT, ou=grid,dc=nexidia,dc=com
objectClass: groupOfUniqueNames
cn: ES_CLIENT
uniqueMember: uid=gateway,ou=grid,dc=nexidia,dc=com

#
# Text Grid Users
#
# Password Bcrypt encoding was generated by https://www.devglan.com/online-tools/bcrypt-hash-generator
#

dn: uid=admin,ou=grid,dc=nexidia,dc=com
objectClass: inetOrgPerson
cn: Admin
sn: TextGrid
userPassword: $2a$04$iL9STzPO2YEqwV0cfF83d.pzNBINUVIIAFVgGJNap/XlpW/wjfwF6

dn: uid=prom,ou=grid,dc=nexidia,dc=com
```

```

objectClass: inetOrgPerson
cn: Prom
sn: TextGrid
userPassword: $2a$04$6HTSvoCFrgnMxjkbJDlcneyrtPltVHhUh30UsxtvcG.Cvqt.Ow16y

dn: uid=defaultuser,ou=grid,dc=nexidia,dc=com
objectClass: inetOrgPerson
cn: DefaultUser
sn: TextGrid
userPassword: $2a$04$05CfQ0RRziWdMy8W5XH4t.Z8PaOOMK78XGnAMPqpUpoTuREwsgBC2

dn: uid=nxttextgrid,ou=grid,dc=nexidia,dc=com
objectClass: inetOrgPerson
cn: nxttextgrid
sn: TextGrid
userPassword: $2a$04$zmj884Gwzh9TGngEngAlseK0ZugLtzhz.uwoHE/BKtYD30GNRYEq2

#
# Reserved service user for connecting to ES. The password is in clear text as ES doesn't do BCrypt encoding.
#
dn: uid=gateway,ou=grid,dc=nexidia,dc=com
objectClass: inetOrgPerson
cn: gateway
sn: TextGrid
userPassword: gateway

```

The syntax and schema of this file is standard based. Care has been taken to use only the most common data constructs to ensure interoperability among different implementations of the LDAP protocol.

The topmost 2 entries define the root container of the Text Grid user store, identified by the `ou=grid, dc=nexidia, dc=com` dn (distinguished name) found in the Gateway/Search service configurations. Groups and users are defined under the root. As stated earlier, the group set must not be changed, but users can be added if necessary, and user credentials can be changed using a third-party LDAP client.

Gateway Windows Integrated Authentication

To enable Windows Integrated Authentication (AD), change the following configuration items to set up the mapping of AD groups and roles. In addition, disable the ldap server and ldap authentication (See [Configuration settings](#) for more details.)

<dataDir>\gateway\etc\application.yml:

```

security:
  windows-authentication.enabled: true
  ## List of roles mapped to 3 levels of access control. Note Windows AD styled groups has escaped back slashes.
  windows-authentication.acl:
    admin-roles: DOMAIN\DEVELOPERS,ADMIN
    user-roles: DOMAIN\DOMAIN USERS,USER

```

Search Node(s) REST Credentials

Security Mode Prerequisite: Installing a Commercial Elasticsearch License

Unlike the outside facing Gateway service, the Search Nodes are Text Grid internal components that are ideally situated in a protected domain accessible only by the Gateway. For this reason security is not enabled when Text Grid is first installed. To activate it, it is necessary to procure a commercial Elasticsearch license.

The following curl command can be used to load a license on a running search instance:

```
curl -X PUT https://<hostname>:9200/_license -H 'content-type:application/json' -d @<path to license file>
```

After the license has been activated, and the `xpack.security.enabled` configuration in `<dataDir>\search\config\elasticsearch.yml` set to `true`, only authenticated users in proper role(s) can interact with the search instance. These users can come from the Elasticsearch native store, or they can be maintained in an outside user credentials store. The current version of Text Grid implements two so-called security realms, LDAP and AD. Each of these can be enabled by changing the respective `enabled` boolean configuration found in the same file.

The procedures for managing Elasticsearch native built-in users (e.g., the `elastic` superuser) are outside the scope of this document. Interested readers are encouraged to consult the relevant Elasticsearch documentation on x-pack security settings.

LDAP Based Authentication:

Note: In case of an external LDAP server being used, please do not use localhost and make sure the external ldap host could be accessed by all search nodes.

```
xpack.security.enabled: true,

## When security is enabled, delegate user authentication to an LDAP server. Map LDAP groups to ES roles
via the role_mapping file.
## Please make sure the value of xpack.security.authc.realms.ldap.nxtg.order is unique among all listed
realms in elasticsearch.yml
xpack.security.authc.realms.ldap.nxtg.enabled: true,
xpack.security.authc.realms.ldap.nxtg.url: "ldap://localhost:8389",
xpack.security.authc.realms.ldap.nxtg.user_search.base_dn: "ou=grid, dc=nexidia, dc=com",
xpack.security.authc.realms.ldap.nxtg.group_search.base_dn: "ou=grid,dc=nexidia,dc=com",
xpack.security.authc.realms.ldap.nxtg.order: '1',
```

As can be seen, this security realm uses Gateway's embedded LDAP service deployed on the same host.

The initial Text Grid installation only has one Search service user provisioned in the LDAP seed data, namely, the `gateway` user assumed by Gateway to interact with Search. Additional users can be added should customer need

arise. The mapping from LDAP groups (e.g., ES_CLIENT) to Elasticsearch roles is done via the `config/role_mapping.yml` file.

AD Based Authentication:

```
xpack.security.enabled: true,

## When security is enabled, delegate user authentication to the Microsoft Active Directory server. Map AD
groups to ES roles via the role_mapping file.

xpack:
  security:
    authc:
      realms:
        active_directory:
          my_ad:
            order: 0
            domain_name: example.com
            url: ldap://1.2.3.4:389
```

The initial Text Grid installation only has one Search service user provisioned in the LDAP seed data, namely, the same service user that the service is installed under, which is used to interact with Search.

For this to work, the `role_mapping.yml` file needs lines similar to

```
superuser:
- "cn=serviceUser,ou=Users,dc=example,dc=com"
```

Note that setting up the `AD roles.yml` file and `elasticsearch.yml` file is highly dependent on the specific setting in the deployed AD, and thus the `treeconfigurer` can not do all this setup, and it must be edited manually after running the `treeconfigurer`.

Once the Search cluster is set up to use AD authentication, you must tell the Gateway service to use this authentication. That is done by taking out the `elasticsearch.credential`, which tells gateway to get the credentials from the Windows Credential Manager under the "textgrid2" key. (This part is done automatically by the `treeconfigurer`).

In order to set this up, use the Credential Manager under the Control Panel, as the service user. Create a Generic Windows Credential "textgrid2", with the username and password of the AD user that should be authenticated.

When credentials are stored in the Windows Credential Manager, then you must pass these values to Powershell scripts using the `-esauth` parameter. The applicable scripts are as follows:



- `ValidateInstall.ps1`
- `SetMaxAnalyzedOffset.ps1`
- `pre_full_cluster_restart_tg2.ps1`

- `post_full_cluster_restart_tg2.ps1`
- `CheckReplicaStatus.ps1`
- `SetReplicaCount.ps1`

Common Exception Protocol (CEP) Server Configuration

Text Grid can be enabled to connect to NICE's CEP Server product. This allows Text Grid to send periodic heartbeats that the CEP server can monitor, as well as sending any error (or in some cases warning) log messages for the CEP server to store. All messages sent to the CEP server are encrypted during transport.

By default, CEP is disabled in Text Grid, but can be enabled using either the default self-signed certificate that the CEP Server creates for clients when it is installed, or a custom (customer-provided) certificate.



To use a custom certificate, that certificate will also have to be installed on the CEP server. This document only covers configuring CEP within Text Grid and installing certificates into Text Grid. Please refer to the CEP Server documentation for its certificate installation instructions.

A Little About Certificates

A certificate is used not only to encrypt the data using its public key (readable only by a receiver that has the matching private key), but also as a means of authenticating that the server is in fact the server that the client is expecting. To provide that, the certificate is signed, and the client validates that signature; however all signatures are not created equal.

Signature chains are used to validate authenticity. For example, the certificate used on a server may have been issued by an organization that is not a trusted root authority, but that organization has been authenticated by another organization, and so on up the chain to a trusted root authority.

- **Self-Signed Certificates:** The lowest-level of authentication is a self-signed certificate, which is not issued by any authority. As its name implies, anyone can create a self-signed certificate, and thus it really provides no guarantee of the server's identity; however it does allow data to be exchanged in an encrypted format. If an organization is extremely confident of its network security and the inability for any unauthorized devices to appear on its network, a self-signed certificate may be sufficient, but some organizations will not be satisfied with a self-signed certificate. The CEP server provides a self-signed certificate to copy to clients, such as Text Grid, for them to use without an organization *needing* to generate their own certificate. An organization could generate their own self-signed certificate and use that, but given that it adds no additional level of security, it's fairly pointless vs. just using the self-signed certificate generated by the CEP server.

- **Internally-Signed Certificates:** The next level of authentication is similar to a self-signed certificate, but is signed within the organization, and keyed specifically to the server on which it is to be installed. That certificate is used for encryption. To authenticate the encryption certificate, that organization then typically installs another certificate of its own – the one it uses to sign its other certificates — on all of its internal computers as a trusted root certificate, even though it's not truly authenticatable through a chain back to a trusted root authority. This type of internally-signed authentication certificate is popular on servers that are only internally-accessible because the outside world does not need to authenticate them up to a trusted root authority, and internally-signed certificates are free, whereas getting a certificate signed by an external trusted authority has a cost associated with it. In this case, the internally-signed certificates for the individual servers authenticate up the chain (usually just one level) to the other certificate that the organization has placed on its computers as a trusted root certificate. Within the organization, client apps are satisfied with authentication. The level of security, however, depends upon how securely that other "trusted" certificate is managed by the organization. To acquire the server-specific signed certificate, some organizations may have an IT Security department that creates the certificates upon request, while others may have the requester create certificate and keypair locally as a certificate signing request (.csr file), and then sent to their IT Security department to be signed and returned as a certificate (.cer file), which will then need to be added to a keystore.
- **Trusted Authority Signed Certificates:** The most secure level of authentication is a certificate that is signed by a trusted signing authority, issued specifically to the server on to which it will be installed. This certificate can be authenticated up the signature chain to a true trusted root authority. A certificate and keypair are created locally as a certificate signing request (.csr file), and then sent to a signing authority to be signed and returned as a certificate (.cer file), which will then need to be added to a keystore. This signing process is not free, and can take time to get, but offer the highest level of authentication. To use this type of certificate for internal servers like Text Grid is uncommon except for those organizations who are extremely concerned with security.

A Little About Keystores

A keystore can contain many certificates and their private keys. Some organizations may put all certificates for their various servers into one keystore, while others may have a separate keystore per server. For the sake of simplification of the instructions below, we'll have a single certificate in the keystore, but that is not a requirement.

There are also multiple formats of keystores, such as JCEKS, JKS, PKCS12 and others, the most common being PKCS12. There are also means to convert between most keystore types, if necessary.

Most convenient and most-secure for Text Grid use is to use Windows' internal, secure Certificate Repository. If a custom certificate is to be used, it can be imported directly into Windows, or it can be provided within its own keystore and imported into Windows from there. (It can also be used directly from that separate keystore without importing it to Windows, but that is not recommended because using a separate keystore will require the password to that keystore to be in clear-text within Text Grid's application.yml file.)



To import the contents of a separate keystore file into Windows, a PKCS12 keystore is required.

Using The Default CEP Client Certificate

The simplest and quickest way to get CEP up and running in Text Grid is to use the certificate that the CEP server created when it was installed. Consult the CEP Server documentation for the location of that certificate, but it will be named `artemis.cer`.

1. Copy the certificate file to anywhere on the Text Grid server. The rest of these steps are all performed on the Text Grid server.
2. Using Windows search in the task bar, type `mmc`, and then choose the "Run as administrator" option.
3. Once the MMC UI opens, in the top menu, click on `File`, then choose `Add/Remove Snap-in...`
4. When the Add or Remove Snap-ins dialog opens, click on the `Certificates` item at the left, then click on `Add >`.
5. When prompted for the type of account, choose `Computer account` then click `Next`.
6. When prompted for the computer to manage, choose `Local computer (the computer this console is running on)` then click `Finish`.
7. You should now see the Add or Remove Snap-ins dialog again. Click `OK` to return back to the main MMC window.
8. At the left, click on `Certificates`.
9. In the center pane will be a list of folders. RIGHT-click on the `Trusted Root Certification Authority` folder and then choose `All Tasks` then `Import`. This will open the Certificate Import Wizard.
10. Verify that the Store Location is set to `Local Machine`, then click `Next`.
11. Click on the `Browse...` button. This will open an "Open" file dialog.
12. Browse to and locate the `artemis.cer` file you copied from the CEP server.
13. Select the `artemis.cer` file, then click `Open`.
14. You will see a dialog that provides some information about the certificate. Click on `Next`.
15. Verify that the `Place all certificates in the following store` item is selected, and that the `Certificate store: value` is `Trusted Root Certification Authority`, then click `Next`.
16. Click `Finish`.
17. It should pop up a message saying "The import was successful." Click `OK`.

18. You should now be back at the main MMC window. In the center pane DOUBLE-click on the [Trusted Root Certification Authorities](#) folder.
19. In the center pane DOUBLE-click on the [Certificates](#) folder. The center pane should fill with a long list of certificates.
20. Find the certificate in the list that is named the same as the CEP server's name.
21. RIGHT-click on that certificate and choose [Properties](#).
22. On the [General](#) tab, near the top, in the [Friendly Name](#) field, enter [CEP-Artemis](#) then click [OK](#).
23. In the list of certificates, for that certificate, you should now see CEP-Artemis under the Friendly Name column. You may now close MMC. If it asks to save the Console settings, you may choose [No](#).

Using Custom Certificates

Importing Custom Trusted Root Certification Authority Certificates

Follow the steps under [Using The Default CEP Client Certificate](#) except specify your custom certificate instead of artemis.cer. If your custom certificate is within a keystore file, at step 12, where you are browsing to the certificate, set the file type (to the right of the [File Name](#) field) to "All Files (.*)" in order to see your keystore, and at step 13, select your keystore file and when you click on [Open](#) you will be prompted to enter the password to your keystore file.

Importing a Dual-Use Certificate

If you have a single, custom certificate that is to be used for both encryption and authenticating the CEP server, see [Importing Custom Trusted Root Certification Authority Certificates](#), above, to install that single certificate.

Importing Encryption Certificates

If using separate certificates for authentication and encryption, import the custom certificate to be used for authentication-only following the steps in [Importing Custom Trusted Root Certification Authority Certificates](#), above, and then install the certificate that is to be used for encryption-only following these steps:

1. Copy the encryption-only certificate file to anywhere on the Text Grid server, or if that certificate is within a keystore, copy that keystore file to anywhere on the Text Grid server. The rest of these steps are all performed on the Text Grid server.
2. Using Windows search in the task bar, type [mmc](#), and then choose the "Open file location" option. This will open a Windows Explorer window, usually in the C:\Windows\System32 directory, and will highlight **mmc.exe**
3. While holding the SHIFT key, RIGHT-click on **mmc.exe**, then choose the "Run as different user" option.

4. When prompted for user credentials, enter the Text Grid *service user's* credentials.
5. Once the MMC UI opens, in the top menu, click on [File](#), then choose [Add/Remove Snap-in...](#)
6. When the Add or Remove Snap-ins dialog opens, click on the [Certificates](#) item at the left, then click on [Add >](#).
7. Click [OK](#) to return back to the main MMC window.
8. At the left, click on [Certificates](#).
9. In the center pane will be a list of folders. RIGHT-click on the [Personal](#) folder and then choose [All Tasks](#) then [Import](#). This will open the Certificate Import Wizard.
10. Verify that the Store Location is set to [Current User](#), then click [Next](#).
11. Click on the [Browse...](#) button. This will open an "Open" file dialog.
12. Click on the drop-down to the right of the [File Name](#) field, and choose "All Files (.*)"
13. Browse to and locate the custom certificate file or the keystore file containing the trusted certificate that file you copied from the CEP server.
14. Select that file, then click [Open](#).
15. You will see a dialog that provides some information about the certificate. Click on [Next](#).
16. Verify that the [Place all certificates in the following store](#) item is selected, and that the [Certificate store: value](#) is [Trusted Root Certification Authority](#), then click [Next](#).
17. You will be prompted to enter the keystore's password. Enter the password in the [Password](#) field, verify that the [Include all extended properties](#) option has a check-mark, then click [Next](#).
18. Verify that the option selected is [Place all certificates in the following store](#) and the [Certificate Store](#) field contains "Personal", then click [Next](#).
19. Click [Finish](#).
20. It should pop up a message saying "The import was successful." Click [OK](#).
21. You should now be back at the main MMC window. In the center pane DOUBLE-click on the [Personal](#) folder.
22. In the center pane DOUBLE-click on the [Certificates](#) folder. The center pane should fill list of personal certificates for the service user. (What you added may be the only one.)
23. Find the certificate in the list that is the one you added to use for Text Grid to encrypt messages being sent to the

CEP server.

24. RIGHT-click on that certificate and choose [Properties](#).
25. On the [General](#) tab, near the top, you will see a [Friendly Name](#) field. This contains the alias to be used by Text Grid to identify that certificate.
 - If there is already a value in the [Friendly Name](#) field, you may keep that, or you may change it to something else.
 - If there is nothing in the Friendly Name field , enter [CEP-Artemis](#) or any name of your choice.
 - Make note of that Friendly Name. You will need it later.
26. Click [OK](#).
27. In the list of certificates, for that certificate, you should now see the pre-existing value or the value you entered under the Friendly Name column. You may now close MMC. If it asks to save the Console settings, you may choose [No](#).
28. Refer to the [Setting CEP Properties In Text Grid](#) section for instructions on how to set properties, and set the following two properties:
 - `cep.keystore.alias=TheFriendlyNameForTheCertificateYouJustAdded`
 - `cep.keystore.type=Windows-MY`

Setting CEP Properties In Text Grid

If using the default certificate provided by the CEP Server, installed exactly as directed in the [Using The Default CEP Client Certificate](#) section, configuration only requires configuring the mandatory CEP properties. Likewise, if a single custom certificate can be used to authenticate the CEP server and encrypt the data being sent to it. If the desire is to have one custom certificate that establishes the issuer of the custom certificates as a trusted authority, and another certificate that holds the encryption keys needed to communicate with the CEP server, then you *might* need to also set the `cep.keyStoreType` property as explained below.

Properties for CEP are set in the `<dataDir>\gateway\etc\application.yml` file. For most of the properties, it is better to adjust these using the treeconfigurer.



For all properties mentioned below, if there is a line in the file beginning with that configuration Key (property name), modify that line; however, unless CEP had been used previously, the lines for the properties will need to be added to the end of the file.

Changes to property values will not be recognized until the Gateway service is restarted.

The format for all properties is:

| key=value

and they are *case-sensitive*

Mandatory Property To Enable CEP



This setting can be set in treeconfigurer. By design, if this setting is not set, then CEP will not be enabled, and any other CEP properties in the application.yml file will be ignored.

cep.broker.hostFQDN=FQDN-of-the-CEP-server

Setting this property enables CEP and tells Text Grid the CEP server's Fully Qualified Domain Name.

Example:

| cep.broker.hostFQDN=myCEPserver.somesubdomain.company.com

Optional CEP Properties



These settings can be set in treeconfigurer

The values shown below are the default values used when the property is *not* specified in the application.yml file.

cep.broker.port=30101

Only if your CEP server uses a custom port number rather than the default port of 30101, use this property to specify that custom port number.

cep.queue.exceptions=TextGrid.Exceptions

This specifies the queue name on which the CEP server expects to receive exceptions from Text Grid.
Needing to set this is very uncommon.

cep.queue.heartbeat=Heartbeats

This specifies the queue name on which the CEP server expects to receive heartbeat messages from Text Grid. Needing to set this is very uncommon.

cep.heartbeatSeconds=60

This specifies how often Text Grid sends heartbeat messages to the CEP server. Any change here will likely need to be coordinated with CEP server settings that indicate how often to expect heartbeats. Needing to set this is very uncommon. Setting this to 0 will disable heartbeats.

cep.exceptionBacklogMaximum=200000

This specifies the maximum number of exception messages that Text Grid should queue up to send to the CEP Server if the CEP server is down / unreachable. The messages will be sent in order once a connection is (re-)established. If there are more exceptions than this value during the time that Text Grid cannot communicate with the CEP server, the oldest messages will be discarded as new ones are added.

cep.verifyHost=true

This specifies whether to verify the CEP server's hostname. If custom certificates are used that do not include the ability to verify the CEP server's FQDN (Fully Qualified Domain Name), OR if DNS lookup is disabled on the Text Grid server, preventing such verification, then this property can be set to false (lower-case) to disable verification in order to allow Text Grid to permit communication with the CEP server without verification. Needing to set this is very uncommon and should be avoided unless absolutely necessary.



Warning! Setting cep.verifyHost to false reduces the level of security by permitting some other, unknown entity to pose as the CEP server, allowing potentially-sensitive data to be acquired.

cep.keystore.alias=CEP-Artemis

This specifies the alias or Friendly Name used to find the encryption certificate in the keystore. (See [Using The Default CEP Client Certificate](#) and [Using Custom Certificates](#).) Needing to set this is very uncommon.

cep.keystore.type=Windows-ROOT

This specifies where the certificate used for encrypting messages to the CEP server is located. If custom certificates are being used and there are separate certificates for providing the encryption keys and providing a trusted signing authority used to validate the encryption certificate, AND if you do not wish to add the encryption certificate as a trusted root authority, then the encryption certificate can be installed into the Personal certificates for the service user (see [Using Custom Certificates](#)), in which case this will need to be set to `Windows-MY`.

+ There are other values that are allowed for special-use cases for quickly testing certificates without importing them into windows. (See [Properties For Special-Use Test Cases](#).)

Properties For Special-Use Test Cases



These settings can **NOT** be set in treeconfigurer.

When using custom certificates, it may be desirable to test those certificates (especially when testing new certificates that will replace soon-to-expire certificates) quickly, without having to import them in to Windows. **These properties are not intended to be used other than for short-term testing, as they expose their test keystore (and possibly truststore) passwords.**

To test, these value would be set in the application.yml during testing, and then deleted from application.yml after testing, or in the case of `cep.keyStoreAlias`, `cep.keyStoreType` and `cep.trustStoreType`, set back to their original values if they had been previously set.

- To test a new encryption certificate, that certificate can be placed in an external keystore file. How to do that goes beyond the scope of this document, but acceptable keystore types are JKS, JCEKS, and the most common being PKCS12.
- To test a new Trusted Root Certification Authority certificate, such as if the encryption certificate being tested requires authority other than one already imported into Windows, or if you need to test a new Trusted Root Certification Authority certificate before replacing an expiring one, that certificate can be placed into an external keystore (referred to as a truststore). How to do that goes beyond the scope of this document, but acceptable keystore types are JKS, JCEKS, and the most common being PKCS12.
- If there is only one certificate to be used for both encryption and certification, then that keystore can be specified as both the keystore and truststore, but unless you intend to use a Trusted Root Certification Authority certificate that has already been imported into windows, you must set both the keystore and truststore properties, below.

Specifying An External Keystore File



These settings can **NOT** be set in treeconfigurer.

cep.keystore.path

Set this to the full path and file name of your test keystore. Do NOT use windows backslashes (\) in the path, and instead use forward slashes (/).

cep.keystore.password

Set this to the password for the test keystore. This would have been set when you created the keystore.

cep.keystore.type

Set this to the type of the test keystore — JKS, JCEKS or PKCS12. If not set, but cep.keystore.path is set, this will default to PKCS12

cep.keystore.alias

Set this to the alias for the encryption certificate contained in the keystore

Specifying An External Truststore File



These settings can **NOT** be set in treeconfigurer

cep.truststore.path

Set this to the full path and file name of your test truststore. Do NOT use windows backslashes (\) in the path, and instead use forward slashes (/).

cep.truststore.password

Set this to the password for the test truststore. This would have been set when you created the truststore.

cep.truststore.type

Set this to the type of the test truststore — JKS, JCEKS or PKCS12. If not set, but cep.truststore.path is set, this will default to PKCS12

Tools

The tools documented in this section are intended to assist with the administration of the Text Grid system.

tg

tg is a command-line administrative tool used to invoke a variety of commands used to manage a Text Grid installation. Secure communication can be configured using the treeconfigurer app. It is invoked using a git-like syntax:

```
tg [-v] [--adminpass <adminpass>] [--adminuser <username>] [--config <config file>] <command> [<args>]
```

Global options appear before the command, while command-specific arguments appear after the command. Global options include:

-v

Verbose mode. Includes additional, non-essential information in the output.

adminpass

Authentication password for the 'admin' user. This is only required if the password for the 'admin' user has been changed from the default.

username

'admin' user. This is only required if the admin user has been changed from the default.

config file

Config file to use. If not given, uses config tgadmin.yml in the data dir.

The following commands are available:

help

Display help information.

```
tg help [<command>]
```

Options:

<command>

An optional command for which to print specific help information.

status

Show the status for each Text Grid service.

```
tg status
```

remove-searchnode

Remove one or more search nodes from service.

```
tg remove-searchnode [--status] <searchNodes>
```

Options

status

Show the status of a removal in progress.

<searchNodes>

A space-separated list of IP addresses for search nodes to remove or view the status.

esindices

```
tg [-v] esindices
```

Show the name, status and sizes of the indices in the search nodes. When in verbose mode also show hidden and system indices.

The health status will be either **GREEN** (fully healthy), **YELLOW** (usable, but the servers are in the process of updating replicas or unable to access all replicas), **RED** (unusable), or **close**. YELLOW status is not a concern if the replicaCount has been recently changed or the service is in the process of doing an automatic reindex. If the status is **close** then this means that the index is no longer being used. See [remove-migrated-indices](#) for more on unused indices.



As of TG 20.1.0, there can be multiple indices for a single doctype/language pair. The indices will be named with a suffix of the form "_xxxxxm" where "xxxxx" is either a number or "az".



As of TG 20.1.0 when gateway restarts, any empty indices will be deleted.

remove-migrated-indices

```
tg remove-migrated-indices
```

When a language or doctype mapping changes, TextGrid will automatically migrate the index into the new form. The old index is left behind in an abundance of caution for possible recovery in the case of a problem. This command deletes these old indices. The [esindices](#) command can be used to show any migrated indices before deletion.



As of TG 20.1.0, when gateway restarts, if the migration occurred more than [\[nexidia.tg.index.closedIndexExpiryDays\]](#) (by default, 30 days) ago, then closed indices that have at least one open index of the same doctype and language will be deleted automatically.

gateway-registry

There is a registry of Gateway nodes that is maintained by the Search Nodes. For normal deployments, this will be the single Gateway node that is part of the deployment. However, for High Availability, there may be multiple Gateway nodes that all access the same Search cluster. Exactly one of the four subcommands is required.

```
tg gateway-registry [--clear] [--deregister <removeHost>] [--list] [--register <addHost>]
```

Subcommands

--clear Clear the registered Gateway nodes.

--deregister Remove the given registered Gateway node (host:port).

--list List the registered Gateway nodes.

--register Add the given Gateway node to the registry ([scheme://host:port](#)).

tg2-info.ps1

Script that creates a machinename.txt file with information about the Text Grid install.

treeconfigurer

[treeconfigurer](#) is a configuration utility that shows and modifies the configuration using a tree structure. It can be run in either command-line-only mode or with a graphical user interface. It will be launched during installation when running Setup-TG.ps1. [treeconfigurer](#) can be launched after setup to change configurations.



The treeconfigurer does more than just setup configuration. It also installs the services. It should be considered a required part of installation rather than just a configuration adjustment tool.

```
treeconfigurer
  [(-h | --help)]
  [(-I <installdrive> | --installdrive <installdrive>)]
  [(-D <datadrive> | --datadrive <datadrive>)]
  [(-b <bindAddr> | --bindaddr <bindAddr>)]
  [(-a <adminGateway> | --admingateway <adminGateway>)]
  [(-l <ldapAddr> | --ldapaddr <ldapAddr>)]
  [{-s <searchAddr> | --searchaddr <searchAddr>}]
  {--seed <seedNode>}
  {--master <initialMasterNode>}
  [--nogateway]
  [--nosearch]
  [--nokibana]
  [--esksowneroverride]
  [--nogui]
  [(-u <serviceUser> | --serviceuser <serviceUser>)]
  [(-U <servicePassword> | --servicepassword <servicePassword>)]
  [--heapsize <heapsize in Gig for search>]
  [--replicas <replica_count>]
  [--snapshotdir <snapshotDirectory>]
  [(-v | --votingonly)]
  [--convertTextToEmoticon <true|false>]
  [--CEPBrokerHostFQDN <CEPBrokerHostFQCN> ]
```

Note that options enclosed in {} indicate options that can occur multiple times. These options can be entered as a list, by listing the option again, or a mix of the two methods. Thus, the following are all equivalent:

```
--master ONE,TWO --master THREE"
"--master ONE --master TWO --master THREE"
"--master ONE,TWO,THREE"
```

-h, --help

Display help information



As of 12.4.2, the -d and -i options which took the entire directory were eliminated in favor of the -D and -l options, which only take the drive letter.

-I <installdrive>, --installdrive <installdrive>

The drive in which to store the TextGrid application. Directory will be <installdrive>:/Program Files/Nexidia/TextGrid2.

-i <installdir>, --installdir <installdir>

(deprecated)

-D <datadrive>, --datadrive <datadrive>

The drive in which to store TextGrid data. Directory will be <datadrive>:/ProgramData/Nexidia/TextGrid2.

-d <datadir>, --datadir <datadir>

(Deprecated)_The directory in which to store TextGrid data, config and logs

-b <bindAddr>, --bindaddr <bindAddr>

The address to use for services on this server (gateway, search and/or ldap)"

-a <adminGateway>, --admingateway <adminGateway>

The address for tgadmin to connect to the gateway (If unspecified, will be bindAddr)

-l <ldapAddr>, --ldapaddr <ldapAddr>

The address for the ldap server

-s <searchAddr>, --bindaddr <searchAddr>

The address of the search cluster (can provide multiple)

--seed <seedNode>

A seed node for the Elasticsearch cluster (can specify multiple seeds)

--master <initialMasterNode>

An initial master node (can specify multiple nodes)

--nogateway

Do not install a gateway service on this server

--nosearch

Do not install a search service on this server

--nokibana

Do not expand the kibana.zip file on this server

--esksowneroverride

Retrieve file owner from a temp file and assign the file owner to elasticsearch.keystore file. Elasticsearch added file owner validation in recent release, the validation could cause text grid search service failed during start up if the service account being used to start search service is also an administrator of search server.

--nogui

Apply configuration without a UI. Requires -I, -D on a new install. Requires -u and -U on all installs. (-U is optional only when user provided with -u is a default system user or Network Service user, in which case it may default to null.)

-u <serviceUser>, --serviceuser <serviceUser>

The user account under which to run TextGrid

-U <servicePassword>, --servicepassword <servicePassword>

The password for the user account under which to run TextGrid

--heapsize <heapsize>

the amount of Java heap space for the search service (in Gig, between 1 and 30)

--replicas <replica_count>

Replica Count for new indices

--snapshotdir <snapshotDirectory>

The directory in which to store Elasticsearch snapshots.

-v, --votingonly

boolean flag indicating whether Elastic node is a dedicated voting only node (default false)

--singleNode

boolean flag indicating whether this is a single node Text Grid server (default false)

--convertTextToEmoticon <boolean>

boolean flag indicating whether to convert specific character patterns to emoticons

--CEPBrokerHostFQDN <FQDN of CEP Broker Host>

String providing the FQDN of the CEP Broker Host. This is required for CEP to function.

Special Considerations for Upgrades

Setting the elasticsearch.connection.requestTimeout to non-0

Elasticsearch recommends setting the elasticsearch.connection.requestTimeout (in <datadir>/gateway/etc/application.yml) to 0. Thus, the default value for new installs is 0. However, in TG 2.0.4 and before, the default value was 5000. In order to fix this issue on upgrades, the treeconfigurer will change 5000 to 0 on upgrades. If you really want

a non-zero value, you must set it to something OTHER than 5000, or it will be changed to 0 the next time treeconfigurer runs.

Changing the installation directory upon upgrade

If the optional—installdrive option is passed on an upgrade, and the provided drive is different from the current installdrive, the upgrade will put the new code in the new drive and delete the old location. Since the configuration and data are under a separate data directory, nothing will be lost.

As of 12.4.2, installations and upgrades can only be done to a canonical location of <installdrive>:/Program Files/Nexidia/TextGrid2. Therefore, if an upgrade is being done to an installation that currently does not reside in the canonical location, and no --installdrive parameter is passed, the installation will be moved to the canonical location on the current install drive.

Changing the data directory upon upgrade

See [moveTextGrid2DataDir.ps1](#)

Sensitivity to Format Change

The elasticsearch.yml and application.yml file formats as read by the Search and Gateway nodes is actually more flexible than the format assumed by the treeconfigurer. Thus, for the two services, the following two configs would be equivalent, but the treeconfigurer will not read them as the same. Thus, it is important to keep the format the same if doing any hand editing of the yml files:

```
section.subsection.key1: "one"
section.subsection.key2: "two"
```

vs

```
section:
  subsection:
    key1: "one"
    key2: "two"
```

Command Line Mode

By using the --nogui option, Treeconfigurer can be used in command-line-only mode to facilitate a scripted installation. At a minimum, the -i, -d and -u options must be provided. Configurer will use the values for the provided options, defaulting as necessary, to modify the configuration files for each Text Grid service. Log information will be written to the console as Configurer runs. Note that there are not enough command line options to support every configuration possibility (e.g. ssl support).

Graphical User Interface Mode

By default, Configurer runs as a GUI application. It presents a tree view of all the configuration settings.

Validate Config

Validating the config does some sanity checking on the install in order to catch common errors. It is not guaranteed to catch everything, but must be run before you can write the configuration to disk.

Write Config

Changes made to the configuration are not written to disk until the Write Config... button is pressed. When the configuration is applied, the configuration files for each service will be changed to match the configuration shown in the tree. Any data directories that do not exist will be created, although the permissions are not adjusted until the Install Services button is pressed.

Install Services

The configuration is not complete until the services are installed. The Install Services dialog prompts for the service user and password and shows the installation of the services.

Install Nodes as Windows Services

This button is only visible if there was a service_list in the command line parameters or when called by Setup with at least one service checked. This button installs the appropriate Windows services. Note that although the services are set to automatic (thus starting automatically upon a reboot), they are only installed and not started by this button. This allows further manual configuration to be completed if necessary before starting the services. The services can be started using standard Windows utilities.

moveTextGrid2DataDir.ps1

```
| moveTextGrid2DataDir.ps1 -datadrive <destination driveletter>
```

As of 12.4.2, it is required that the datadir be in a specific directory (\ProgramData\Nexidia\TextGrid2) with only the drive letter as a variable. However, installs of 2.0.4 and earlier did not have this restriction, so it is possible that someone wants to upgrade an installation that is not in the canonical location. This script is used to move the datadir from the original location to the canonical location.

Before starting the script, stop all services, following the recommendations for how to stop an elasticsearch cluster.

After running the script, before the services can be used, the treeconfigurer must be run and the services must be reinstalled (either as part of an upgrade, or just as by starting the treeconfigurer manually).

ValidateInstall.ps1

Run this simple script to do a quick sanity check of your TG2 install. It will check the Elasticsearch license, add a document, search for that document, get that document, and then delete the test document.

PrepareTG2Upgrade.ps1

This script assists with doing an upgrade from TG1.1 to TG20.x. Full instructions are in the comments at the beginning of the script, but here is a summary:

1. Copy this script to the machine which has the TG1 storage node installed.
2. Ensure that the Storage and Control nodes are running, but the Workflow and Search Nodes are not running.
3. Run this script. It will prompt you for various pieces of information about your desired TG20.x install and produce two scripts (installTG2Upgrade.ps1 and createMigrateYaml.ps1).
4. Copy the InstallTG2Upgrade.ps1 script to each TG20.x machine in the same directory where the Readme and the "Setup-TG.ps1" script are. Run this script on each machine. It will prompt, walk you through the installation of TG20.x, and start the services.
5. Run <installDir>/apps/bin/ValidateInstall.ps1 to do a quick sanity check on the installation.
6. Run "createMigrateYaml.ps1" on the TG20.x machine with gateway installed.
7. Do the migration using either migrate.cmd or migrate_service.bat
8. Once fully validated, TG1 can be uninstalled.

migrate

`migrate` is a utility for migrating from Text Grid 1.1 to Text Grid 20.x. It can be used to migrate on the same machine, or between two systems.



While you can use the information below to do the upgrade from TG 1.1 to TG 20.x, the provided [PrepareTG2Upgrade.ps1](#) script will assist in the migration. This section should still be read for troubleshooting, optimization, and general understanding of the process.

Usage

```
| migrate [--config=<configFile>]
```

--config=<configFile>

A YAML configuration file for the migration.

Configuration File

The following describes the contents of the migration configuration file. Settings with default values are optional, so may

be commented out.

```
# The number of threads to use to write to the TG2 Storage cluster. Default: 1
threadCount: <NUMBER>
# The number of documents to include in each request to the TG2 Storage
Cluster. Default: 100
batchSize: <NUMBER>
# Whether or not to only migrate missing documents. Default: false
repair: <BOOLEAN>
# optional parameter which skips the determination of how many documents are
in the TG1 database
# migrateCount: 0

# optional parameter to only migrate a certain type of document (general,
email, or chat)
# documentType: chat
# optional parameter to only migrate a certain language of document (en, es,
etc...)
# language: en

# The location of a csv file with the list of ids.
# If set, then the migration uses a new method introduced in 12.4.4
# If set to a file that does not exist, it will be generated.
# If set to an existing file (created with dsbulk or from a previous run),
it uses this as the
# state of which ids have already been migrated and which need to be.
idFile: <STRING>
# Number of threads to use to read from the TG1 cassandra database if idFile
is set. Default: 1
srcThreadCount: <NUMBER>
# Size of the batch of documents to read from the TG1 database if idFile is
set. Default: 1000
srcBufferSize: <NUMBER>

# The source block specifies how to connect to the Text Grid 1.1 Storage
Service.
source:
    # Required. The host name or IP address of the Storage Service.
    host: <STRING>
    # The port on which the Storage Service is listening. Default: 9042
    port: <NUMBER>
    # The minimum ID to migrate (can be used to avoid moving obsolete
documents). default: 0
    minimumId: <NUMBER>

    # The keyspace used by the Storage Service. Default: ntxtgrid
    keyspace: <STRING>
    security:
        # Required. The credentials for accessing the Storage Service.
```

```

credentials:
  user: "jyrus"
  pass: ***** # replace ***** with actual password
  # Whether or not to use secure communication (SSL/TLS) with the
Storage Service. Default: false
  tlsEnabled: <BOOLEAN>
  # If tlsEnabled is true, the keystore and truststore blocks are
required. Otherwise, they may be omitted. Note that the keystore and
truststore may be the same.
  keystore:
    # The full path to the keystore file.
    storeFile: <STRING>
    # The format of the keystore file, one of JKS, JCEKS, or PKCS12.
    storeType: <STRING>
    # The password used to secure the keystore file.
    storePass: <STRING>
  truststore:
    # The full path to the truststore file.
    storeFile: <STRING>
    # The format of the truststore file, one of JKS, JCEKS, or PKCS12.
    storeType: <STRING>
    # The password used to secure the truststore file.
    storePass: <STRING>

# The dest block specifies how to connect to the Text Grid 2 Gateway Service.
dest:
  # Required. The host name or IP address of the Gateway Service.
  host: <STRING>
  # The port on which the Gateway Service is listening. Default: 28002
  port: <NUMBER>
  security:
    # Required: The credentials for accessing the Gateway Service.
    credentials:
      user: "defaultuser"
      pass: ***** # replace ***** with actual password
    # Whether or not to use secure communication (SSL/TLS) with the
Gateway Service. Default: false
    tlsEnabled: <BOOLEAN>
    # Whether or not a self-signed certificate is used by the Gateway
Service. Note that self-signed certificates should not be used in production
deployments. Default: false
    lenientSslValidation: <BOOLEAN>
    # If tlsEnabled is true AND lenientSslValidation is true, the keystore
and truststore blocks are required. Otherwise, they may be omitted. Note
that the keystore and truststore may be the same.
    keystore:
      # The full path to the keystore file containing the self-signed
certificate.
      storeFile: <STRING>

```

```
# The format of the keystore file, one of JKS, JCEKS, or PKCS12.
storeType: <STRING>
# The password used to secure the keystore file.
storePass: <STRING>
truststore:
# The full path to the truststore file.
storeFile: <STRING>
# The format of the truststore file, one of JKS, JCEKS, or PKCS12.
storeType: <STRING>
# The password used to secure the truststore file.
storePass: <STRING>
```

Before installation of Text Grid 20.x

Run `tg -v esindices` command on existing Text Grid 1 to get number of documents to be migrated to new Text Grid 20.x

Stop all the Text Grid 1.1.x services except Control and Storage.

Do not uninstall Text Grid 1.1.x until after the migration is complete.

If the destination machine is the same as the source machine, then ensure that the Text Grid 20.x `<installdir>` and `<datadir>` are distinct from the Text Grid 1.1.x `<installdir>` and `<datadir>`, and there is sufficient disk space to do the migration.

Running the migrate app

Once Text Grid 20.x is installed and configured, ensure the following services are running:

- Text Grid 1.1:
 - Control
 - Storage
- Text Grid 20.x:
 - Gateway
 - Search
- Note that the Text Grid 1.1 Workflow and Search nodes should NOT be running.

Create a configuration file for the migration. See [Configuration File](#) for details about this file. From a command prompt, run the `migrate` app as described in [Usage](#). Output will be sent to the console, with a checkpoint printed periodically for monitoring progress. In addition, a log file will be created in the `<datadir>/logs/migrate` directory. You can also check

the current status using the REST api <http://127.0.0.1:9250/checkpoint>.

If the migrate app is stopped before completion, it may be rerun, to no ill effect, as any documents that were already migrated will be migrated again, as an update, rather than an add. If the migrate app is running with the `idFile` property set, see [mergeNewIdsFile](#) for information on doing a secondary run. If the optional language and/or `documentType` properties are set when running with the `idFile` option, the resulting csv file will show any documents that were not already marked as migrated and are skipped due to not matching these properties as "error", rather than "migrated". This will allow a secondary run to be done with different values for the language and/or `documentType` and migrate these skipped documents.

If any documents are added to the Text Grid 2 database before the migrate app is run, they will be retained, as long as the ids are not duplicates of a doc in the migration. If the doc id is duplicated, it will be replaced by whichever add/update is run last.

Any document that reached a terminal state other than "indexed" (e.g. "processing_failed", "indexing_failed") state will not be migrated, and an error will be reported. These documents are typically not of concern, as they were not searchable in Text Grid 1. For any of these failures that occur, the number and percentage of each of these failure types will be reported.

Migrate as a Service

The script `<installdir>/apps/bin/migrate_service.bat` can be run to set up a service to run migration.

```
| migrate_service.bat [install|remove] service_user service_password
```

The config file MUST be installed in `<datadir>\apps\etc\migrate.yaml`.

Once this service is installed, migration can be run by starting the service.

When the migration is complete, the service will exit.

Migration progress can be monitored using the checkpoint API (<http://127.0.0.1:9250/checkpoint>) or by looking in the `<datadir>/logs/migrate` directory for the log.

Troubleshooting the migration service startup is best done by consulting the `<datadir>/logs/migrate` directory for logs.

Using the idFile property

As of TG 12.4.4, the new optional `idFile`, `srcBatchSize`, and `srcThreadCount` properties were introduced and made the default method of migration.

```
| idFile: <fileName>.csv
| srcBatchSize: 1000
| srcThreadCount: 4
```

If that file exists, then migrate will use it as a list of the ids to migrate. If it does not exist, then migrate will create it and

then use it as a list of ids to migrate.

Format of the file is:

```
id,state
13,indexed,migrated
12,unprocessed
11,indexed,error
```

The first line is a header. The first column is the ids, the second is the state in Cassandra, and if there is a third, it indicates that the document has already been migrated or ran into an error that migrate was unable to recover from.

The file can either be created by the migrate app or by dsbulk. dsbulk is a tool that can be downloaded from <https://downloads.datastax.com/#bulk-loader> and should be unzip/installed. Set JAVA_HOME to <TG1InstallDir>/jre. Run the dsbulk command from <dsbulk installation folder>\bin with parameters like shown below:

 Only use versions older than 1.11.0, as 1.11.0 and later do not work with the version of Cassandra in TG 1.1.

 dsbulk must be installed into a path that does not have spaces.

```
dsbulk.cmd unload -u jyru -p Z3pp3l1n -h <ip> -query "SELECT id,state FROM ntxtgrid.docs WHERE
state='indexed' ALLOW FILTERING" > allTG1Ids.csv
```

OR

```
dsbulk.cmd unload -u jyru -p Z3pp3l1n -h <ip> -query "SELECT id,state FROM ntxtgrid.docs" > allTG1Ids.csv
```

The first command will only list the docs that need migrating but may time out on larger systems.

The second command should work on larger systems.

The results of the migration will be updated in an output file "<filename>-new".

 If a migration is stopped part way through or there are errors, consult the section [mergeNewIdsFile](#) for handling.

Timeouts

If the migration process fails due to time-out errors, please follow steps outlined in Wiser [KB0093248](#)

In case the additional timeout setting recommended in KB does not help (typically due to a very large number of documents), set the migrateCount property in the migrate.yaml file in one of the following ways:

 If you are migrating using the [PrepareTG2Upgrade.ps1](#) script, this file is auto generated when running the [createMigrateYaml.ps1](#) script. It is located in the <datadir>/apps/etc folder.

1. Set migrateCount in the migrate.yaml file to 0. migrate will use an arbitrary 1000000 as the number of documents. Note that it will not stop at this arbitrary cutoff but the percentages in the checkpoints will be misleading. When an arbitrary number like this is selected, once the migration has been run, the actual calculated number of documents will be updated in the migrate.yaml file so that if a repair needs to be run, the status percentages will be more accurate.

```
| migrateCount: 0
```

2. Set migrateCount to the value returned by the following dsbulk command:

```
| dsbulk-1.10.0\bin>dsbulk.cmd count -u jyru -p Z3pp3l1n -k ntxtgrid -t docs -h <rpc_address>
```

Optimization



If you are using the `PrepareTG2Upgrade.ps1` generated migration script, the `migrate.yaml` file will already be modified to use the recommended threadCount and 1000 for the batchSize.

In cases where migration process appears to be slow and not meeting SLAs, increase the other settings in migrate.yaml for threadCount and batchSize. We currently recommend setting the total of threadCount and srcThreadCount to half of the number of physical cores on your system.

For example, a sample dry run on a large dataset to be migrated on a physical box w/ 96gb of ram and 32 logical processors was taking over 24 hours with the default settings. Increasing threadCount to 8 and batchSize to 1000 resulted in reducing the time to within half an hour for migration of same dataset. Note that if you have already started a migration and decide you want to change these settings, it will start the migration completely over, but you do not need to delete the documents that have already been migrated.

Post Migration Validation

Verify that for each of these migrations, a very high percentage of the documents that were in the TG1 Elasticsearch database have been moved to the TG2 Elasticsearch database. The migration progress can be monitored by checking the number of documents in TG2 and comparing it with the number previously determined from TG1 using the `tg -v esindices` command.



As of TG 20.1.0, indices can be split by the values in the ids, so a single large index (e.g. `nxtg_chat_en_5_3`) in TG 1.1.x may be broken into multiple indices in TG 20.x (e.g. `nxtg_chat_en_6_3_0m`, `nxtg_chat_en_6_3_1m`, `nxtg_chat_en_6_3_azm`). The total number of documents in all these indices should be close to the number of documents in the single TG 1.1 equivalent index.

One case where the number of documents will not match is if any of the documents in the TG1 storage database were in the "deleting" state. This means that the document was still in the TG1 Elasticsearch database even though it should have been deleted, and thus will not be migrated over.

If NOT using the idFile method and the count of documents between the TG1 elasticsearch database and TG2 elasticsearch database differs by too much, you can re-run the migrate app with the added "repair: true" option in the

yaml file. This will go through and re-migrate only the missing documents. Depending on what percentage of documents succeeded on the initial migration, the resulting repair migration may be faster or slower than the original one. If you switch to using the `idFile` property after trying a run without that property, then the repair option may help, but if the initial run was using the `idFile` property, then the repair option does not provide any additional speedup.



The count of documents referred to by migrate is the count of documents in the Storage cluster. This number may differ considerably from the number of documents in the Elasticsearch database, as chats are stored in Elasticsearch in such a way that they show up as multiple documents in any Elasticsearch count.

Before using the new Text Grid 20.x deployment, carefully check the logs for any errors and run queries on the old and new installations to ensure that all the docs have been transferred. For example, if one of the services is halted while the migrate is in process, a 500 internal server error can be generated along with an error message "Failed to add docs". If you are using the `idFile` option, you can get more detailed info on the state of individual documents by looking at the `idfile-new` file.

Disable the TG 1.1.x services.

Optional Post migration cleanup

Uninstall Text Grid 1.1.x.

Delete the Text Grid 1.1.x `<installdir>` and `<datadir>`

mergeNewIdsFile

Usage:

```
| mergeNewIdsFile.cmd <idfile>
```

During migration, if the `idFile` property is set, there is an `<idFile>.csv` file that represents the state of the migration at the beginning. After the migrate has run, there is a `<idFile>.csv-new` file that represents what occurred during the migration. If the migration is run successfully to the complete migration, but there were some errors due to communication issues with the services, then this `<idFile>.csv-new` file may be used as the input `idFile` for a secondary run to finish up these ids that did not migrate on the first try.

However, if the migration is stopped or errors out part way through, then use [mergeNewIdsFile](#) to combine the `idFile.csv` and `idFile.csv-new` into the actual state of the migration, which will be output as `<idFile>.csv-merged`. This merged file can then be used as the `idFile` for a secondary run to complete the migration.

Kibana

Kibana is a very powerful application developed by Elasticsearch that supports a lot of different capabilities. For TextGrid, we will focus only on two capabilities: the use of Kibana to send REST commands directly to the Elastic Cluster (Search Nodes), and the use of Kibana to provide Disaster Recovery through Cross Cluster Replication.

Configuration

Basic Kibana configuration is handled through the [treeconfigurer](#) application. Additional configuration can be done by editing the kibana.yml file stored in the <datadir>/kibana/config directory.

Starting and running Kibana

Start Kibana by running the kibana.cmd file in the <installDir>/apps/bin directory. Once it is running, you connect to the Kibana UI by pointing your browser to <http://127.0.0.1:5601>.

Executing a cURL command using Kibana

From the Kibana UI, click on Dev Tools. This is available both under the three bar Settings dropdown under "Management" or next to the wrench icon near the bottom right of the opening page. From here, you can create and execute REST commands against the Elasticsearch Cluster.

Implementing Disaster Recovery using Cross Cluster Replication

In order to access the Cross Cluster Replication Management section of Kibana, pull up the Management Section in the left icon toolbar. From there, select Cross Cluster Replication. Although Cross Cluster Replication provides a rich set of operations, we will use a simple subset to provide Disaster Recovery for our Search cluster. The Gateway is stateless, so no additional DR is needed for it other than setting up a backup Gateway that accesses the secondary Search cluster.

Setup for Disaster Recovery



For more information on Cross Cluster Replication, consult <https://www.elastic.co/guide/en/elasticsearch/reference/current/ccr-getting-started.html>.

Create two clusters.

In the primary cluster, add at least one document of each and type and language that will be stored (e.g. if English and Spanish are supported, and all three document types (general, email, and chat) will be included, then add 6 documents that represent the cross-section).

Using Kibana, set the primary data center to be a leader, with all indices as leader indices, with the secondary data center indices as followers.

At this point, the secondary cluster will start automatically copying any changes made into the primary cluster over to the secondary cluster.

Read-only Usage during Primary Outage

Simply point NIA at the backup Gateway, which is pointing to the secondary data center. This should only be used for read access, as any documents added to the secondary cluster will not get automatically ingested back into the primary

cluster once it comes back up.

Full failover

Use Kibana to covert followers to a normal index.

Point NIA to the backup gateway.

Failback

For failback, there is no simple mechanism for picking back up and making only the updates on the original that were needed so several steps are needed:

1. Set up the original primary as a follower of the secondary
2. Convert the original primary to a leader
3. Reconfigure the backup as a follower

-
1. <https://www.elastic.co/guide/en/elasticsearch/reference/current/modules-cluster.html#disk-based-shard-allocation>
 2. <https://www.elastic.co/guide/en/elasticsearch/reference/current/cluster-update-settings.html>
 3. <https://www.elastic.co/guide/en/elasticsearch/reference/current/settings.html#dynamic-cluster-setting>